



Programmation Python : API



LUDIKSCIENCES

API (Application Programming Interface)

2 systèmes qui échangent des informations doivent trouver un langage commun

=> Interface de communication

=> Ensemble normalisé de classes, de méthodes, de fonctions et de constantes qui sert de façade

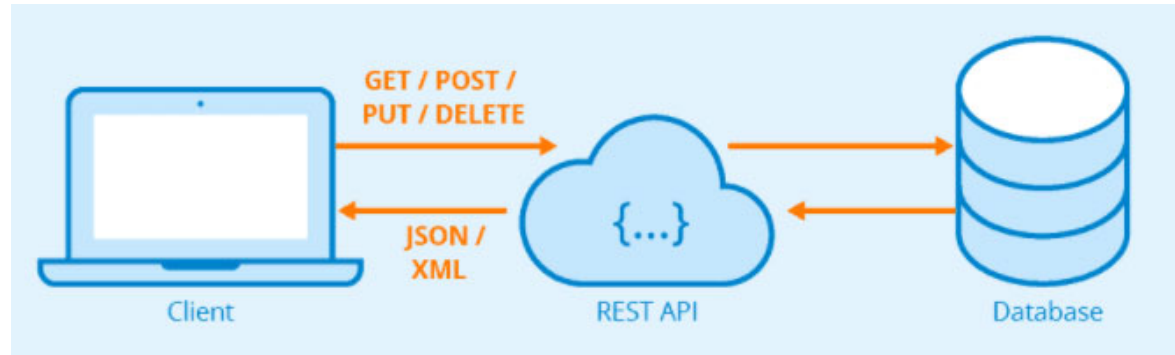
C'est une architecture Client/Serveur

=> Nécessité d'avoir une documentation sur les appels autorisés

=> Possibilité de restreindre les accès et/ou de tracer les demandes



WEB et API REST



REST (representational state transfer) est une norme d'architecture pour développer des services Web (2000)

- Client–serveur
- Absence de conservation d'état entre 2 appels
- Gestion de l'obsolescence des données (Cache)
- Façade, on ne peut accéder au serveur de données
- Uniformité d'interface (URL d'appel, retour JSON ou XML)



Requests: HTTP for Humans™

Installation

```
$ python -m pip install requests
```

Usage

```
import requests

rq = requests.get("http://monip.org")
print(rq.text)
```

Ex : Passage de paramètres

```
par = {'key1': 'val1', 'key2': 'val2'}
rq = requests.get("http://monip.org/GET" \
                  , params= par)

print(rq.url)
```

```
http://monip.org/GET?key1=val1&=val2
```

Méthodes pour requêtes HTTP : Get, Put, Delete, Options

Possède un parseur intégré Json : `rq.json()`

Statut de la demande : `rq.status_code ==> 200`



Format JSON

JSON (JavaScript Object Notation)

est un format d'échange de données

- ensembles de paires « nom » (alias « clé ») / « valeur »
- listes ordonnées de valeurs

Module standard Json en Python

Parcourir

Pour convertir un objet Python en json

```
{  
  "email": {  
    "A": "toi@societe.fr",  
    "CC": "lui@societe.fr",  
    "PJ": [  
      { "titre": "document.doc"},  
      { "titre": "image.jpg"},  
      { "titre": "tableur.xls"}  
    ]  
  }  
}
```

```
import json
```

```
monjson = json.loads(data)  
print(monjson["email"]["PJ"][1]["titre"])
```

```
image.jpg
```

```
x = {"fname": "Mickey", "lname": "Mouse"}  
unjson = json.dumps(x)
```

Projet API 1/2

Créez une interface en mesure d'afficher les trajets des lignes de métro :

1) Consulter le format de données renvoyées par l'adresse :

<https://gist.githubusercontent.com/aliou/065689914b6677cfb06c/raw/a0d973e81c3ff4fbea706950ccdb01c555f66600/metro-lines.json>

2) Créer un module metro.py en mesure d'accéder aux données et d'afficher le contenu (import Request) – affichage brut

3) A l'aide du module json (import json), faire une fonction **Affichage** en mesure de présenter les données de la manière suivante :

Métro 1 La Defense / Chateau de Vincennes

Métro 2 ...



Projet API 2/2

4) Créer une fonction recherche en mesure d'afficher le trajet d'une seule ligne de métro

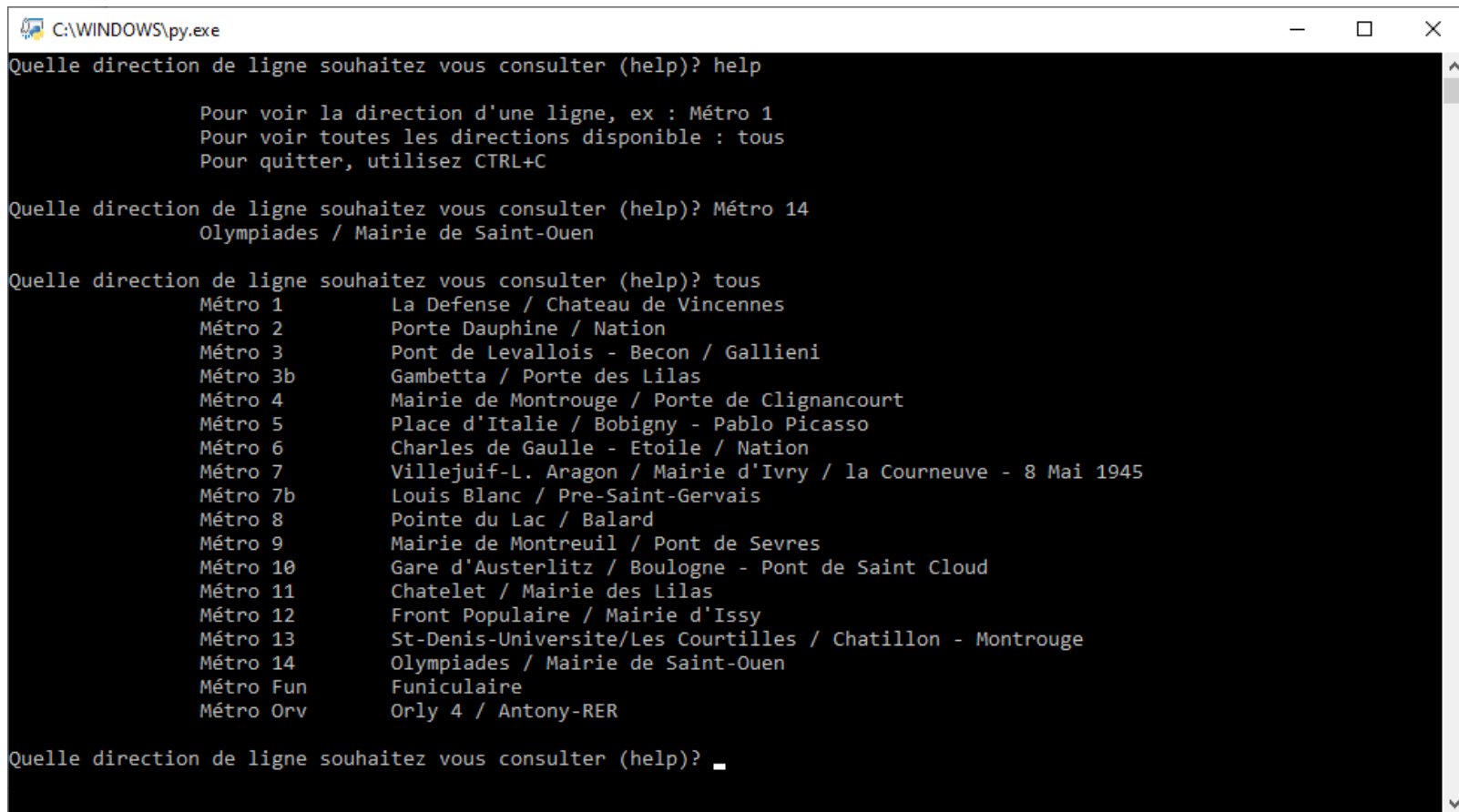
5) Implémenter une interface utilisateur (input) permettant d'interroger :

- Aide sur les saisies
- Trajet d'une ligne en fonction de la saisie utilisateur, ex : Métro 14
- Tous les trajets si l'utilisateur saisi : tous

Optionnel : Récupérer le fichier de données à partir de l'url et remplacer dans le code par un appel au fichier pour une version hors ligne



Projet API : Interface attendue



```
C:\WINDOWS\py.exe
Quelle direction de ligne souhaitez vous consulter (help)? help

    Pour voir la direction d'une ligne, ex : Métro 1
    Pour voir toutes les directions disponible : tous
    Pour quitter, utilisez CTRL+C

Quelle direction de ligne souhaitez vous consulter (help)? Métro 14
    Olympiades / Mairie de Saint-Ouen

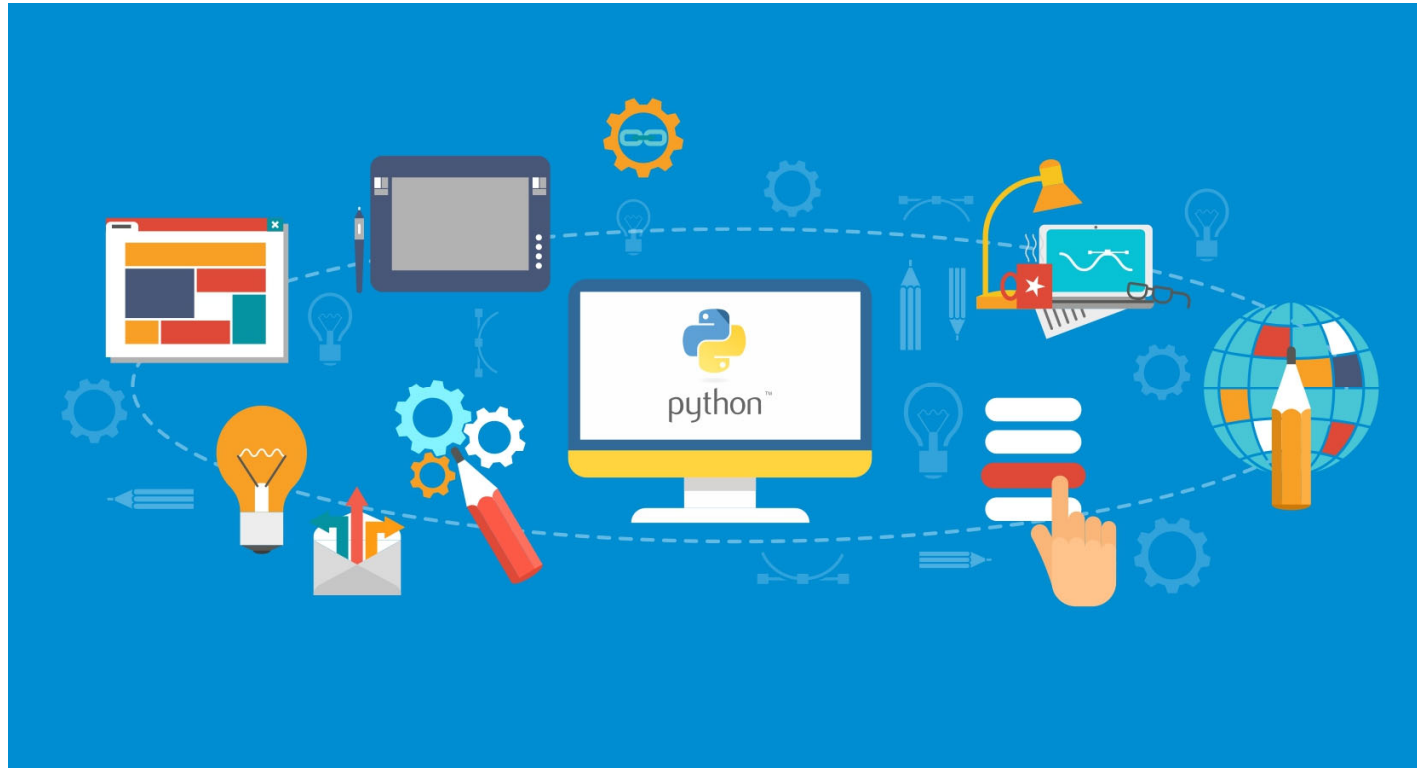
Quelle direction de ligne souhaitez vous consulter (help)? tous
    Métro 1      La Defense / Chateau de Vincennes
    Métro 2      Porte Dauphine / Nation
    Métro 3      Pont de Levallois - Becon / Gallieni
    Métro 3b     Gambetta / Porte des Lilas
    Métro 4      Mairie de Montrouge / Porte de Clignancourt
    Métro 5      Place d'Italie / Bobigny - Pablo Picasso
    Métro 6      Charles de Gaulle - Etoile / Nation
    Métro 7      Villejuif-L. Aragon / Mairie d'Ivry / la Courneuve - 8 Mai 1945
    Métro 7b     Louis Blanc / Pre-Saint-Gervais
    Métro 8      Pointe du Lac / Balard
    Métro 9      Mairie de Montreuil / Pont de Sevres
    Métro 10     Gare d'Austerlitz / Boulogne - Pont de Saint Cloud
    Métro 11     Chatelet / Mairie des Lilas
    Métro 12     Front Populaire / Mairie d'Issy
    Métro 13     St-Denis-Universite/Les Courtilles / Chatillon - Montrouge
    Métro 14     Olympiades / Mairie de Saint-Ouen
    Métro Fun    Funiculaire
    Métro Orv    Orly 4 / Antony-RER

Quelle direction de ligne souhaitez vous consulter (help)? _
```

NB : L'utilisateur ne doit pas être obligé de relancer le programme entre chaque demande.

(facultatif) Vous pouvez implémenter un choix Quitter





Programmation Python : Webserveur



LUDIKSCIENCES

Un serveur WEB

Dans IDLE, créer un fichier serveur.py
Et insérer le code suivant :

```
import http.server
import socketserver

MonHandler = http.server.SimpleHTTPRequestHandler

with socketserver.TCPServer("", 8080), MonHandler) as httpd:
    print("serving at port", 8080)
    httpd.serve_forever()
```

Copier le fichier mapage.html
Dans le même répertoire que la script

A l'aide d'un navigateur WEB, aller à :

```
http://localhost:8080/mapage.html
```

Alternative : 127.0.0.1:8080/mapage.html



LUDIKSCIENCES

Projet Serveur WEB 1/2

- 1) Vous devrez créer une classe hérité de `http.server.SimpleHTTPRequestHandler`
- 2) En surchargeant, la méthode **do_GET**, vous explorerez plusieurs opportunités du webserveur en fonction de l'URL saisie (chemin) :

A. La construction d'une page html dynamique :

Par exemple, en saisissant **127.0.0.1:8080/tapage.html**
une réponse 200 sera envoyée au navigateur, elle sera suivie d'un header
puis d'un code HTML au format UTF-8:

```
<html><body>Ceci est une page dynamique !</body></html>
```

B. Le routage d'une URL particulière vers une page existante :

Par exemple, en saisissant **127.0.0.1:8080/secret**
une réponse 200 sera envoyée au navigateur, elle sera suivie d'un header
puis un fichier html copié vers le navigateur (ex : mapage.html)



Projet Serveur WEB 2/2

C. Une API simplifiée

Par exemple, en saisissant **127.0.0.1:8080/API**
une réponse 200 sera envoyée au navigateur, elle sera suivie d'un header
puis du contenu d'un fichier Json (ex: metros.json)

D. Protection du répertoire source

Par exemple, en saisissant **127.0.0.1:8080/**
une réponse 404 sera envoyée au navigateur au lieu du listing des fichiers présents

E. Fonctionnement nominal

Si une adresse ex : **127.0.0.1:8080/mapage.html** est saisie dans le navigateur, le fichier
devra continuer à pouvoir s'afficher, y compris pour les sous-répertoires.

ex : 127.0.0.1:8080/dossier/mapage.html
correspondra à un fichier situé dans le répertoire
C:/repertoireuserveur/dossier/mapage.html

